

РОССИЙСКИЙ УНИВЕРСИТЕТ ДРУЖБЫ НАРОДОВ

Факультет физико-математических и естественных наук

Кафедра прикладной информатики и теории вероятностей

ОТЧЕТ

ПО ЛАБОРАТОРНОЙ РАБОТЕ № 6

дисциплина: Архитектура компьютера —

Студент: Ромеро Гаспар Фернандо Алексей

Группа: НКАбд-02-25

МОСКВА

2025 г.

Оглавление

Цель заботы

Теоретическое введение

2.1 Адресация в NASM

2.2 Арифметические операции в NASM

2.3 Перевод символа числа в десятичную символьную запись

Выполнения лабораторной работы

3.1 Символьные и численные данные в NASM

3.2 Выполнение арифметических операций в NASM

Выполнение самостоятельной работы

Выводы

Список литературы

Цель работы

Освоение арифметических инструкций языка ассемблера NASM

Теоретическое введение

2.1 Адресация в NASM

В NASM режим адресации определяет способ доступа к операндам в памяти. Основные режимы включают:

- **Непосредственная адресация** (`mov eax, 42`), где операнд является константой.
- **Регистровая адресация** (`mov ebx, eax`), использующая регистры напрямую.
- **Прямая адресация в память** (`mov ecx, [0x8048000]`), обращающаяся к абсолютному адресу.

NASM требует использования квадратных скобок `[ ]` для доступа к памяти, что отличает его от ссылок на непосредственные метки.

Расширенные режимы адресации обеспечивают большую гибкость:

- **Косвенная адресация через регистр** (`mov eax, [ebx]`), где регистр содержит адрес.
- **Базовый + смещение** (`mov ecx, [ebp-8]`), используется для локальных переменных в стеке.
- **Индексная адресация** (`mov edx, [array + esi*4]`) для работы с массивами, где `esi` — это индекс, а `4` — размер элемента.

Также поддерживается **RIP-относительная адресация** (`mov rax, [rel variable]`), что критически важно для позиционно-независимого кода (PIC), так как позволяет вычислять адреса относительно регистра RIP.

NASM поддерживает комбинации режимов адресации, такие как **база + индекс * масштаб + смещение** (`mov eax, [ebx + esi*4 + 16]`), что идеально подходит для структур и многомерных массивов. Масштаб может быть 1, 2, 4 или 8. Регистры общего назначения (R8-R15) также могут использоваться в адресации.

Важно помнить, что в NASM инструкция `mov eax, метка` загружает *адрес* метки, а `mov eax, [метка]` загружает *содержимое* по этому адресу. Для работы с внешними данными используются директива `extern` и соответствующая адресация.

Выравнивание данных влияет на производительность; директива `align 16` обеспечивает оптимальное расположение в памяти.

РЕЖИМ АДРЕСАЦИИ	ПРИМЕР СИНТАКСИСА	ОПИСАНИЕ	ТИПИЧНОЕ ИСПОЛЬЗОВАНИЕ
НЕПОСРЕДСТВЕННЫЙ	<code>mov eax, 42</code>	Операнд является константой	Константы, маски
ПРЯМОЙ В ПАМЯТЬ	<code>mov ebx, [0x400000]</code>	Обращение к абсолютному адресу	Нижняя память, порты ввода-вывода
КОСВЕННЫЙ ЧЕРЕЗ РЕГИСТР	<code>mov ecx, [edx]</code>	Регистр содержит адрес	Простые указатели
БАЗА + СМЕЩЕНИЕ	База + смещение	База (EBP/RBP) + смещение	Локальные переменные в стеке
ИНДЕКС * МАСШТАБИРОВАНИЕ	<code>mov edx, [array + esi*4]</code>	Массив + индекс * размер	Однородные массивы
БАЗА + ИНДЕКС	<code>mov ebx, [eax + ecx]</code>	База + индекс (без масштабирования)	Строки, буферы
БАЗА + ИНДЕКС * МАСШТАБ + СМЕЩЕНИЕ	<code>mov eax, [ebx+esi*8+20]</code>	Полный режим для структур	Поля структур
RIP-ОТНОСИТЕЛЬНЫЙ	<code>mov rax, [rel var]</code>	Адрес относительно	Независимый от положения код (PIC),

(X86-64)		RIP	глобальные переменные
РЕГИСТРОВЫЙ	add eax, ebx	Оба операнда в регистрах	Быстрые вычисления

2.2 Арифметические операции в NASM

NASM поддерживает основные арифметические операции с помощью специальных инструкций набора команд: `ADD` (сложение), `SUB` (вычитание), `INC` (инкремент), `DEC` (декремент), `MUL` (умножение без знака), `IMUL` (умножение со знаком), `DIV` (деление без знака) и `IDIV` (деление со знаком). Эти операции работают с операндами размером 8, 16, 32 или 64 бита и напрямую влияют на флаги регистра EFLAGS/RFLAGS, в частности на `CF` (флаг переноса), `OF` (флаг переполнения), `ZF` (флаг нуля) и `SF` (флаг знака). Например, инструкция `ADD EAX, EBX` прибавляет значение из `EBX` к `EAX`, сохраняет результат в `EAX` и обновляет соответствующие флаги.

У операций умножения и деления есть особые характеристики. `MUL` и `IMUL` работают с одним неявным операндом: для 8-битного умножения `AL` умножается на операнд, и 16-битный результат сохраняется в `AX`; для 16 бит: `AX` × операнд → `DX:AX`; для 32 бит: `EAX` × операнд → `EDX:EAX`; и для 64 бит: `RAX` × операнд → `RDX:RAX`.

`DIV` и `IDIV` следуют аналогичному, но обратному шаблону: они делят делимое в `DX:AX` (16 бит), `EDX:EAX` (32 бита) или `RDX:RAX` (64 бита) на операнд, сохраняя частное в `AL`/`AX`/`EAX`/`RAX` и остаток в `AH`/`DX`/`EDX`/`RDX`.

Крайне важно убедиться, что регистры `EDX`/`RDX` правильно подготовлены (расширены с помощью `CDQ` или `CQO` для знаковых операций) перед выполнением деления.

Кроме основных операций, NASM включает такие инструкции, как `ADC` (сложение с переносом), `SBB` (вычитание с заёмом), `NEG` (получение дополнительного кода) и `CMP`

(сравнение, которое представляет собой вычитание без сохранения результата). Для операций расширенной многократной точности (сложение/вычитание чисел размером 128+ бит) используются последовательности с `ADC`/`SBB`. Флаги критически важны для последующего управления потоком выполнения.

Инструкции `CBW`, `CWD`, `CDQ` и `CQO` расширяют знак для подготовки операндов.

ИНСТРУКЦИЯ	СИНТАКСИС	ОПЕРАЦИЯ	ЗАДЕЙСТВОВАННЫЕ РЕГИСТРЫ	ЗАТРАГОВАННЫЕ ФЛАГИ
ADD		приёмник = приёмник		CF, OF
	ADD приёмник, источник	приёмник + источник		
SUB		приёмник = приёмник - источник		CF, OF
	SUB приёмник, источник	приёмник - источник		
INC		операнд = операнд + 1		OF, ZF (CF)
	INC операнд	операнд + 1		
DEC		операнд = операнд - 1		OF, ZF (CF)
	DEC операнд	операнд - 1		
MUL	MUL источник	AX = AL × источник (8 бит)	AX/DX:AX/EDX:EAX/RDX:RAX	CF, OF
		DX:AX = AX × источник (16 бит)		опреде
		EDX:EAX = EAX × источник (32 бит)		
		RDX:RAX = RAX × источник (64 бит)		
IMUL	IMUL источник	Умножение со знаком (варианты: 1, 2	приёмник или пара DX:AX/EDX:EAX/RDX:RAX	CF, OF
	IMUL приёмник, источник			опреде

		IMUL приёмник, или 3 операнда) источник1, источник2			
DIV	DIV источник	$AL = AX \div AL/AH, \quad AX/DX, \quad EAX/EDX, \quad \text{Все не}$ источник, $AH = RAX/RDX$ остаток (8 бит) $AX = DX:AX \div$ источник, $DX =$ остаток (16 бит) $EAX = EDX:EAX$ $\div$ источник, EDX = остаток (32 бит) $RAX =$ $RDX:RAX \div$ источник, $RDX =$ остаток (64 бит)			
IDIV	IDIV источник	Деление со $AL/AH, \quad AX/DX, \quad EAX/EDX, \quad \text{Все не}$ знаком RAX/RDX (аналогично DIV)			
ADC	ADC приёмник, источник	приёмник = приёмник приёмник + источник + CF			CF, OF
SBB	SBB приёмник, источник	приёмник = приёмник приёмник - источник - CF			CF, OF
NEG	NEG операнд	операнд = - операнд операнд (дополнительный код)			CF, OF
CMP	CMP операнд1, операнд2	операнд1 - Никакие операнд2 (без сохранения			CF, OF

2.3 Перевод символа числа в десятичную символьную запись

Преобразование числового символа (символа ASCII) в десятичное представление на ассемблере включает преобразование символов, таких как '0'-'9' (значения ASCII 48-57), в их целочисленные числовые эквиваленты (0-9). Это достигается путём вычитания значения ASCII символа '0' (48 в десятичной системе, 0x30 в шестнадцатеричной). Например, чтобы преобразовать символ '7' (ASCII 55) в его десятичное значение 7, вычисляется $55 - 48 = 7$. Эта операция важна для обработки пользовательского ввода, чтения текстовых файлов, содержащих числа, или интерпретации аргументов командной строки в строковом формате.

Для преобразования полной строки числовых символов в целочисленное значение реализуется итеративный алгоритм, который обрабатывает каждую цифру слева направо:

1. Инициализировать результат значением 0.
2. Для каждого символа:
 - Проверить, находится ли он в диапазоне от '0' до '9'.
 - Вычесть '0', чтобы получить числовое значение цифры.
 - Умножить накопленный результат на 10.
 - Прибавить новую цифру.
3. Обработать необязательные знаки ('+' или '-') в начале строки.

Этот алгоритм использует позиционную десятичную запись: $"123" = ((1 \times 10) + 2) \times 10 + 3$. В ассемблере для этого применяются инструкции `'SUB'`, `'MUL'/'IMUL'`, `'ADD'` и сравнения с границами ASCII.

Типичная реализация в NASM использует цикл, который проходит по строке, преобразует каждый символ и накапливает результат в регистре. Критически важно проверять диапазон каждого символа (ASCII 48-57), чтобы избежать преобразования нечисловых символов, а также обнаруживать переполнение (overflow), когда число превышает размер регистра. Также необходимо учитывать завершение строки (нулевой символ, 0).

ЭЛЕМЕНТ	ЗНАЧЕНИЕ/ПРЕДСТАВЛЕНИЕ	ОПЕРАЦИЯ ПРЕОБРАЗОВАНИЯ	ПРИМЕР
СИМВОЛ '0'	ASCII 48 (0x30)	Значение = ASCII - 48	'0' → 48 - 48 = 0
СИМВОЛ '5'	ASCII 53 (0x35)	Значение = ASCII - 48	'5' → 53 - 48 = 5
СИМВОЛ '9'	ASCII 57 (0x39)	Значение = ASCII - 48	'9' → 57 - 48 = 9
СТРОКА "123"	'1'(49), '2'(50), '3'(51)	Итерация: результат = (результат × 10) + цифра	((0×10+1)×10+2)×10+3 = 123
ЗНАК МИНУС	Символ '-' (ASCII 45)	Установить флаг, обработать цифры, инвертировать результат	"-45" → флаг_отрицательный = 1, значение = 45, результат = -45
ЗНАК ПЛЮС	Символ '+' (ASCII 43)	Пропустить, обрабатывать цифры как обычно	" + 27 " → пропустить '+', значение = 27
ПРЕДЕЛ ДОПУСТИМОЙ ЦИФРЫ МНОЖИТЕЛЬ ОСНОВАНИЯ	ASCII 48-57 включительно	Проверить: символ ≥ '0' && символ ≤ '9'	'A'(65) → недопустимо
	Основание 10	Для каждой новой позиции умножить накопленное значение × 10	Десятки: ×10, сотни: ×100 и т.д.
ПРЕОБРАЗОВАНИЕ БЕЗ УМНОЖЕНИЯ	Для систем без MUL	×10 = (значение×8) + (значение×2) = (значение<<3) + (значение<<1)	5×10 = (5<<3)+(5<<1)=40+10=50
ШЕСТНАДЦАТЕРИЧНОЕ ПРЕОБРАЗОВАНИЕ	'0'-'9', 'A'-'F', 'a'-'f'	'0'-'9': -48, 'A'-'F': -55, 'a'-'f': -87	'A'(65)→65-55=10, 'f'(102)→102-87=15
ЭФФЕКТИВНАЯ ПРОВЕРКА	Альтернатива сравнениям	двум SUB символ,'0'; CMP результат,'9'; JA ошибка	'7'→49-48=1, 1≤9 → допустимо
ДЕСЯТИЧНОЕ В ASCII (ВЫВОД)	Обратное преобразование	Делить на 10, сохранять остатки, + '0', выводить в обратном порядке	123→123/10=12(остаток3)→'3', 12/10=1(остаток2)→'2', 1→'1'→"123"

- **Преобразование без умножения:** Для систем без инструкции MUL использовать повторное сложение или сдвиги: $\times 10 = (\text{значение} \times 8) + (\text{значение} \times 2) = (\text{значение} \ll 3) + (\text{значение} \ll 1)$.
- **Преобразование шестнадцатеричных чисел:** Символы '0'-'9' (вычесть 48), 'A'-'F' (вычесть 55), 'a'-'f' (вычесть 87).
- **Эффективная проверка:** Вместо двух сравнений использовать: SUB символ, '0', затем CMP результат, 9 и JA ошибка.
- **Преобразование десятичного числа в ASCII (для вывода):** Последовательно делить на 10, сохранять остатки (цифры), преобразовывать в ASCII путём сложения с '0', выводить в обратном порядке.

Выполнения лабораторной работы

Создайте директорию для программ лабораторной работы № 6, перейдите в неё и создайте файл lab6-1.asm. Запишите символ "6" в регистр eax (mov eax, "6") и символ "4" в регистр ebx (mov ebx, "4"). Затем добавьте значение регистра ebx к значению регистра eax (add eax, ebx; сумма записывается в регистр eax). После этого выведите результат. Однако для использования функции sprintLF регистр eax должен иметь адрес, поэтому используйте дополнительную переменную. Запишите значение регистра eax в переменную buf1 (mov [buf1], eax) через 7 секунд, затем запишите адрес переменной buf1 в регистр eax (mov eax, buf1) и вызовите функцию sprintLF.

```
fernando@ubuntu25-10:~/work/arch-pc/lab06$ nano lab6-1.asm
fernando@ubuntu25-10:~/work/arch-pc/lab06$ nasm -f elf lab6-1.asm
fernando@ubuntu25-10:~/work/arch-pc/lab06$ ld -m elf_i386 -o lab6-1 lab6-1.o
fernando@ubuntu25-10:~/work/arch-pc/lab06$ ./lab6-1
j
fernando@ubuntu25-10:~/work/arch-pc/lab06$
```

Когда вы ожидаете увидеть число 10 при выводе значения регистра eax, фактическим результатом будет символ «j». Это происходит потому, что код символа «6» в двоичном коде равен 00110110 (или 54 в десятичном), а код символа «4» — 00110100 (или 52 в десятичном).

При выполнении команды «add eax, ebx» результатом будет сумма кодов: 01101010 (или 106 в десятичном), что соответствует символу «j». Затем вы изменяете текст программы и записываете в регистры числа вместо символов.

```
fernando@ubuntu25-10:~/work/arch-pc/lab06$ nano lab6-1.asm
fernando@ubuntu25-10:~/work/arch-pc/lab06$ nasm -f elf lab6-1.asm
fernando@ubuntu25-10:~/work/arch-pc/lab06$ ld -m elf_i386 -o lab6-1 lab6-1.o
fernando@ubuntu25-10:~/work/arch-pc/lab06$ ./lab6-1

fernando@ubuntu25-10:~/work/arch-pc/lab06$
```

При запуске программы вы не получите число 10. Вместо этого выводится символ с кодом 10, который обозначает символ конца строки (возврат каретки). Этот символ не отображается на консоли, но добавляет пустую строку.

Как упоминалось ранее, файл in_out.asm реализует процедуры преобразования символов ASCII в числа и наоборот.

Преобразуйте текст программы, используя эти функции.

```
fernando@ubuntu25-10:~/work/arch-pc/lab06$ nano lab6-2.asm
fernando@ubuntu25-10:~/work/arch-pc/lab06$ nasm -f elf lab6-2.asm
fernando@ubuntu25-10:~/work/arch-pc/lab06$ ld -m elf_i386 -o lab6-2 lab6-2.o
fernando@ubuntu25-10:~/work/arch-pc/lab06$ ./lab6-2
106
```

Результатом работы программы является число 106. Команда 'add' складывает коды символов «б» и «4» ($54 + 52 = 106$). Однако, в отличие от предыдущей программы, функция 'iprintLF' выводит число, а не символ, код которого она представляет. Теперь заменим символы на цифры. Функция iprintLF выводит число, и операндами являются числа (а не коды символов). Следовательно, вывод — 10. Функция printf заменяется на iprint. Затем создается

и запускается исполняемый файл. Результат отличается тем, что в нем отсутствуют переносы строк.

```
fernando@ubuntu25-10:~/work/arch-pc/lab06$ nano lab6-2.asm
fernando@ubuntu25-10:~/work/arch-pc/lab06$ nasm -f elf lab6-2.asm
fernando@ubuntu25-10:~/work/arch-pc/lab06$ ld -m elf_i386 -o lab6-2 lab6-2.o
fernando@ubuntu25-10:~/work/arch-pc/lab06$ ./lab6-2
10
fernando@ubuntu25-10:~/work/arch-pc/lab06$
```

```
fernando@ubuntu25-10:~/work/arch-pc/lab06$ nano lab6-2.asm
fernando@ubuntu25-10:~/work/arch-pc/lab06$ nasm -f elf lab6-2.asm
fernando@ubuntu25-10:~/work/arch-pc/lab06$ ld -m elf_i386 -o lab6-2 lab6-2.o
fernando@ubuntu25-10:~/work/arch-pc/lab06$ ./lab6-2
10fernando@ubuntu25-10:~/work/arch-pc/lab06$
```

3.2 Выполнение арифметических операций в NASM

В качестве примера выполнения арифметических операций в NASM мы представляем программу для вычисления арифметического выражения:

$$f(x) = (5 * 2 + 3) / 3$$

```
fernando@ubuntu25-10:~/work/arch-pc/lab06$ touch lab6-3.asm
10fernando@ubuntu25-10:~/work/arch-pc/lab06$ nano lab6-3.asm
fernando@ubuntu25-10:~/work/arch-pc/lab06$ nasm -f elf lab6-3.asm
fernando@ubuntu25-10:~/work/arch-pc/lab06$ ld -m elf_i386 -o lab6-3 lab6-3.o
fernando@ubuntu25-10:~/work/arch-pc/lab06$ ./lab6-3
Результат: 4
Остаток от деления: 1
fernando@ubuntu25-10:~/work/arch-pc/lab06$
```

Измените программный код, чтобы вычислить выражение:

$$f(x) = (4 * 6 + 2) / 5$$

Создайте исполняемый файл и протестируйте его.

```
fernando@ubuntu25-10:~/work/arch-pc/lab06$ nano lab6-3.asm
fernando@ubuntu25-10:~/work/arch-pc/lab06$ nasm -f elf lab6-3.asm
fernando@ubuntu25-10:~/work/arch-pc/lab06$ ld -m elf_i386 -o lab6-3 lab6-3.o
fernando@ubuntu25-10:~/work/arch-pc/lab06$ ./lab6-3
Результат: 5
Остаток от деления: 1
```

В качестве еще одного примера рассмотрим программу для вычисления решения задачи с использованием идентификационного номера студента.

В этом случае число, которое нужно вычислить, вводится с клавиатуры. Ввод с клавиатуры является символическим, и для корректной работы арифметических операций NASM символы необходимо преобразовать в числа. Это можно сделать с помощью функции `atoi` из файла `in\_out.asm`. Создайте исполняемый файл и запустите его. Проверьте вывод программы, аналитически рассчитав номер варианта. Включите ответы на следующие вопросы в свой лабораторный отчет:

1. Какие строки в листинге 6.4 отвечают за отображение сообщения «Ваш вариант:»?

Эти две инструкции загружают адрес строки `get` (содержащей "Ваш вариант:") в регистр `eax` и вызывают подпрограмму `sprint` для ее отображения на экране.

2. Для чего используются следующие инструкции?

`mov ecx, x` -> Загружает адрес буфера `x` в регистр `ecx`.

`mov edx, 80` -> Устанавливает максимальную длину входных данных (80 байт) в регистре `edx`.

call sad -> Вызывает подпрограмму чтения строк с клавиатуры.

3. Каково назначение инструкции «call atoi»?

Инструкция `'call atoi'` вызывает подпрограмму, которая преобразует строку ASCII в целое число. Она преобразует строковое представление идентификационного номера студента в числовое значение, которое сохраняется в регистре `'eax'`.

4. Какие строки в листинге 6.4 отвечают за вычисление варианта?

`xor edx, edx` ; Обнуление `edx` для деления

`mov ebx, 20` ; Загрузка делителя (количество вариантов)

`div ebx` ; Деление: `eax/20`, остаток в `edx`

`inc edx` ; Увеличение остатка на 1 (варианты от 1 до 20)

5. В какой регистр записывается остаток от деления при выполнении инструкции «div ebx»?

При выполнении инструкции `'div ebx'` остаток от деления записывается в регистр `'edx'`. Инструкция `'div'` делит 64-битное число в паре регистров `'edx':eax` на операнд (`'ebx'`).
Результат:

- Частное записывается в `'eax'`
- Остаток записывается в `'edx'`

6. Каково назначение инструкции «inc edx»?

Инструкция ``inc edx`` увеличивает значение в регистре ``edx`` на 1. В контексте этой программы: после ``div ebx`` регистр ``edx`` содержит остаток от деления (0-19). ``inc edx`` преобразует этот остаток в номер варианта (1-20), поскольку варианты нумеруются с 1, а не с 0.

7. Какие строки в листинге 6.4 отвечают за отображение результата вычисления?

Эти инструкции загружают вычисленный номер варианта из регистра `edx` в регистр `eax` и вызывают подпрограмму `iprintLF`, которая отображает номер на экране, за которым следует перенос строки.

```
fernando@ubuntu25-10:~/work/arch-pc/lab06$ nano variant.asm
fernando@ubuntu25-10:~/work/arch-pc/lab06$ nasm -f elf variant.asm
fernando@ubuntu25-10:~/work/arch-pc/lab06$ ld -m elf_i386 -o variant variant.o
fernando@ubuntu25-10:~/work/arch-pc/lab06$ ./variant
Введите N студенческого билета:
1132255520
Ваш вариант: 1
fernando@ubuntu25-10:~/work/arch-pc/lab06$
```

Выполнение самостоятельной работы

Написать программу вычисления выражения $y = f(x)$. Программа должна выводить выражение для вычисления, выводить запрос на ввод значения x , вычислять заданное выражение в зависимости от введенного x , выводить результат вычислений. Вид функции $f(x)$ выбрать из таблицы 6.3 вариантов заданий в соответствии с номером полученным при выполнении лабораторной работы. Создайте исполняемый файл и проверьте его работу для значений x_1 и x_2 из 6.3. (вариант 11)

```
fernando@ubuntu25-10:~/work/arch-pc/lab06$ touch lab6-4.asm
fernando@ubuntu25-10:~/work/arch-pc/lab06$ nano lab6-4.asm
fernando@ubuntu25-10:~/work/arch-pc/lab06$ nasm -f elf lab6-4.asm
fernando@ubuntu25-10:~/work/arch-pc/lab06$ ld -m elf_i386 -o lab6-4 lab6-4.o
fernando@ubuntu25-10:~/work/arch-pc/lab06$ ./lab6-4
Вычисление выражения:  $y = 10(x + 1) - 10$ 
Введите значение x: 1
Результат:  $y = 10$ 
fernando@ubuntu25-10:~/work/arch-pc/lab06$ ./lab6-4
Вычисление выражения:  $y = 10(x + 1) - 10$ 
Введите значение x: 7
Результат:  $y = 70$ 
fernando@ubuntu25-10:~/work/arch-pc/lab06$ █
```

Выводы

Четыре программы были успешно реализованы на языке ассемблера NASM, что продемонстрировало владение основными концепциями низкоуровневого программирования. В ходе этих упражнений участники приобрели навыки работы с регистрами процессора (EAX, EBX, ECX, EDX), выполнения базовых арифметических операций, преобразования типов данных и управления вводом/выводом с помощью библиотеки `in_out.asm`. Программы, скомпилированные и выполненные корректно, подтвердили понимание процесса ассемблерного программирования: от написания исходного кода до генерации исполняемого файла с использованием NASM и компоновщика LD, тем самым закрепив необходимые навыки для разработки на архитектуре x86.

Список литературы

1. Intel Corporation. (2024). *Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 2: Instruction Set Reference*.
<https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html>
2. NASM Documentation. (2023). *NASM Manual Chapter 3: The NASM Language*.
<https://www.nasm.us/doc/nasmdoc3.html>
3. Stack Overflow. (2023). *Addressing modes in NASM*.
<https://stackoverflow.com/questions/tagged/nasm+addressing-mode>
4. GeeksforGeeks. (2023). *Arithmetic Instructions in 8086 Microprocessor*.
<https://www.geeksforgeeks.org/arithmetic-instructions-8086-microprocessor/>
5. NASM Documentation. (2023). *NASM Manual: Chapter 5 - Standard Macro Packages*.
<https://www.nasm.us/doc/nasmdoc5.html>
6. GeeksforGeeks. (2023). *Convert a string to integer in assembly language*.
<https://www.geeksforgeeks.org/convert-string-integer-assembly-language/>
7. Stack Overflow. (2023). *String to integer conversion in x86 assembly*.
<https://stackoverflow.com/questions/19309749>
- 8.